

[illegible]

```

0,      1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0,
0,      0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
0,      0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0,
0,      1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1,
1,      1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1,
0,      1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
1,      1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
0,      1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
1,      1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1,
1,      1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,
1,      1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,      1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1)),
'frame': None, 'target_names': array(['malignant', 'benign'],
dtype='<U9'), 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer
wisconsin (diagnostic) dataset\
n-----\n\n**Data Set
Characteristics:**\n\n: Number of Instances: 569\n\n: Number of
Attributes: 30 numeric, predictive attributes and the class\n\n
: Attribute Information:\n      - radius (mean of distances from center
to points on the perimeter)\n      - texture (standard deviation of
gray-scale values)\n      - perimeter\n      - area\n      - smoothness
(local variation in radius lengths)\n      - compactness (perimeter^2 /
area - 1.0)\n      - concavity (severity of concave portions of the
contour)\n      - concave points (number of concave portions of the
contour)\n      - symmetry\n      - fractal dimension ("coastline
approximation" - 1)\n\n      The mean, standard error, and "worst" or
largest (mean of the three\n      worst/largest values) of these
features were computed for each image,\n      resulting in 30 features.
For instance, field 0 is Mean Radius, field\n      10 is Radius SE,
field 20 is Worst Radius.\n\n      - class:\n      - WDBC-
Malignant\n      - WDBC-Benign\n\n: Summary Statistics:\n\n
n===== \n
Min      Max\n===== \n
radius (mean):          6.981  28.11\ntexture (mean):
9.71  39.28\nperimeter (mean):          43.79  188.5\narea
(mean):          143.5  2501.0\nsmoothness (mean):
0.053  0.163\ncompactness (mean):          0.019  0.345\
nconcavity (mean):          0.0  0.427\nconcave points

```

```

(mean):          0.0    0.201\nsymmetry (mean):
0.106  0.304\nfractal dimension (mean):          0.05    0.097\
nradius (standard error):          0.112  2.873\ntexture (standard
error):          0.36    4.885\nperimeter (standard error):
0.757  21.98\narea (standard error):          6.802  542.2\
nsmoothness (standard error):          0.002  0.031\ncompactness
(standard error):          0.002  0.135\nconcavity (standard error):
0.0    0.396\nconcave points (standard error):          0.0    0.053\
nsymmetry (standard error):          0.008  0.079\nfractal dimension
(standard error):  0.001  0.03\nradius (worst):
7.93   36.04\ntexture (worst):          12.02  49.54\
nperimeter (worst):          50.41  251.2\narea (worst):
185.2  4254.0\nsmoothness (worst):          0.071  0.223\
ncompactness (worst):          0.027  1.058\nconcavity
(worst):          0.0    1.252\nconcave points (worst):
0.0    0.291\nsymmetry (worst):          0.156  0.664\
nfractal dimension (worst):          0.055  0.208\
n===== \n\nMissing
Attribute Values: None\n\nClass Distribution: 212 - Malignant, 357 -
Benign\n\nCreator: Dr. William H. Wolberg, W. Nick Street, Olvi L.
Mangasarian\n\nDonor: Nick Street\n\nDate: November, 1995\n\nThis is
a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.\n
nhttps://goo.gl/U2Uwz2\n\nFeatures are computed from a digitized image
of a fine needle\naspirate (FNA) of a breast mass. They describe\
ncharacteristics of the cell nuclei present in the image.\n\
nSeparating plane described above was obtained using\nMultisurface
Method-Tree (MSM-T) [K. P. Bennett, "Decision Tree\nConstruction Via
Linear Programming." Proceedings of the 4th\nMidwest Artificial
Intelligence and Cognitive Science Society,\npp. 97-101, 1992], a
classification method which uses linear\nprogramming to construct a
decision tree. Relevant features\nwere selected using an exhaustive
search in the space of 1-4\nfeatures and 1-3 separating planes.\n\nThe
actual linear program used to obtain the separating plane\nin the 3-
dimensional space is that described in:\n[K. P. Bennett and O. L.
Mangasarian: "Robust Linear\nProgramming Discrimination of Two
Linearly Inseparable Sets",\nOptimization Methods and Software 1,
1992, 23-34].\n\nThis database is also available through the UW CS ftp
server:\n\nftp ftp.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-
learn/WDBC/\n\n.. dropdown:: References\n\n - W.N. Street, W.H.
Wolberg and O.L. Mangasarian. Nuclear feature extraction\n for
breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on\n
Electronic Imaging: Science and Technology, volume 1905, pages 861-
870,\n San Jose, CA, 1993.\n - O.L. Mangasarian, W.N. Street and
W.H. Wolberg. Breast cancer diagnosis and\n prognosis via linear
programming. Operations Research, 43(4), pages 570-577,\n July-
August 1995.\n - W.H. Wolberg, W.N. Street, and O.L. Mangasarian.
Machine learning techniques\n to diagnose breast cancer from fine-
needle aspirates. Cancer Letters 77 (1994)\n 163-171.\n',
'feature_names': array(['mean radius', 'mean texture', 'mean

```

```

perimeter', 'mean area',
    'mean smoothness', 'mean compactness', 'mean concavity',
    'mean concave points', 'mean symmetry', 'mean fractal
dimension',
    'radius error', 'texture error', 'perimeter error', 'area
error',
    'smoothness error', 'compactness error', 'concavity error',
    'concave points error', 'symmetry error',
    'fractal dimension error', 'worst radius', 'worst texture',
    'worst perimeter', 'worst area', 'worst smoothness',
    'worst compactness', 'worst concavity', 'worst concave points',
    'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
'filename': 'breast_cancer.csv', 'data_module':
'sklearn.datasets.data'}

X = breast_cancer_data.data
y = breast_cancer_data.target
feature_names = breast_cancer_data.feature_names
print(feature_names)

['mean radius' 'mean texture' 'mean perimeter' 'mean area'
'mean smoothness' 'mean compactness' 'mean concavity'
'mean concave points' 'mean symmetry' 'mean fractal dimension'
'radius error' 'texture error' 'perimeter error' 'area error'
'smoothness error' 'compactness error' 'concavity error'
'concave points error' 'symmetry error' 'fractal dimension error'
'worst radius' 'worst texture' 'worst perimeter' 'worst area'
'worst smoothness' 'worst compactness' 'worst concavity'
'worst concave points' 'worst symmetry' 'worst fractal dimension']

```

Q2. convert dataset to pandas dataframe.

```

df_features = pd.DataFrame(X, columns=feature_names)
print(df_features.head())

```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
--	------------------	----------------	---------------------	-----------------

0	0.27760	0.3001	0.14710
0.2419			
1	0.07864	0.0869	0.07017
0.1812			
2	0.15990	0.1974	0.12790
0.2069			
3	0.28390	0.2414	0.10520
0.2597			
4	0.13280	0.1980	0.10430
0.1809			

	mean fractal dimension	...	worst radius	worst texture	worst perimeter \
0	0.07871	...	25.38	17.33	
184.60					
1	0.05667	...	24.99	23.41	
158.80					
2	0.05999	...	23.57	25.53	
152.50					
3	0.09744	...	14.91	26.50	
98.87					
4	0.05883	...	22.54	16.67	
152.20					

	worst area	worst smoothness	worst compactness	worst concavity \
0	2019.0	0.1622	0.6656	0.7119
1	1956.0	0.1238	0.1866	0.2416
2	1709.0	0.1444	0.4245	0.4504
3	567.7	0.2098	0.8663	0.6869
4	1575.0	0.1374	0.2050	0.4000

	worst concave points	worst symmetry	worst fractal dimension
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

Q3. Append dataframe containing tumor features with diagnostic outcomes.

```
df_target = pd.Series(y, name='diagnosis')
df = pd.concat([df_features, df_target], axis=1)
print(df.head())
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	
0.11840					

1	20.57	17.77	132.90	1326.0
0.08474				
2	19.69	21.25	130.00	1203.0
0.10960				
3	11.42	20.38	77.58	386.1
0.14250				
4	20.29	14.34	135.10	1297.0
0.10030				

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	
0.2419				
1	0.07864	0.0869	0.07017	
0.1812				
2	0.15990	0.1974	0.12790	
0.2069				
3	0.28390	0.2414	0.10520	
0.2597				
4	0.13280	0.1980	0.10430	
0.1809				

	mean fractal dimension	...	worst texture	worst perimeter	worst area \
0	0.07871	...	17.33	184.60	
2019.0					
1	0.05667	...	23.41	158.80	
1956.0					
2	0.05999	...	25.53	152.50	
1709.0					
3	0.09744	...	26.50	98.87	
567.7					
4	0.05883	...	16.67	152.20	
1575.0					

	worst smoothness	worst compactness	worst concavity	worst concave points \
0	0.1622	0.6656	0.7119	
0.2654				
1	0.1238	0.1866	0.2416	
0.1860				
2	0.1444	0.4245	0.4504	
0.2430				
3	0.2098	0.8663	0.6869	
0.2575				
4	0.1374	0.2050	0.4000	
0.1625				

	worst symmetry	worst fractal dimension	diagnosis
0	0.4601	0.11890	0

1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

Q4.Find relation in diagnosis by using mean.

```
mean_features = df.groupby('diagnosis').mean()
print(mean_features)
```

	mean radius	mean texture	mean perimeter	mean area \
diagnosis				
0	17.462830	21.604906	115.365377	978.376415
1	12.146524	17.914762	78.075406	462.790196

	mean smoothness	mean compactness	mean concavity \
diagnosis			
0	0.102898	0.145188	0.160775
1	0.092478	0.080085	0.046058

	mean concave points	mean symmetry	mean fractal dimension
...			
diagnosis			
...			
0	0.087990	0.192909	0.062680
...			
1	0.025717	0.174186	0.062867
...			

	worst radius	worst texture	worst perimeter	worst
area \				
diagnosis				
0	21.134811	29.318208	141.370330	1422.286321
1	13.379801	23.515070	87.005938	558.899440

	worst smoothness	worst compactness	worst concavity \
diagnosis			
0	0.144845	0.374824	0.450606
1	0.124959	0.182673	0.166238

	worst concave points	worst symmetry	worst fractal
dimension			
diagnosis			
0	0.182237	0.323468	

```
0.091530
1          0.074444          0.270246
0.079442

[2 rows x 30 columns]
```

Q5. create to dataframes - one for positive, one for negative(diagnosis).

```
df_positive = df[df['diagnosis'] == 1]
df_negative = df[df['diagnosis'] == 0]
print("Positive diagnosis (Malignant):")
print(df_positive.head())
print("\nNegative diagnosis (Benign):")
print(df_negative.head())
```

Positive diagnosis (Malignant):

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
19	13.540	14.36	87.46	566.3	0.09779
20	13.080	15.71	85.63	520.0	0.10750
21	9.504	12.44	60.34	273.9	0.10240
37	13.030	18.42	82.61	523.8	0.08983
46	8.196	16.84	51.71	201.9	0.08600

	mean compactness	mean concavity	mean concave points	mean symmetry \
19	0.08129	0.06664	0.047810	0.1885
20	0.12700	0.04568	0.031100	0.1967
21	0.06492	0.02956	0.020760	0.1815
37	0.03766	0.02562	0.029230	0.1467
46	0.05943	0.01588	0.005917	0.1769

	mean fractal dimension	...	worst texture	worst perimeter	worst area \
19	0.05766	...	19.26	99.70	711.2
20	0.06811	...	20.49	96.09	630.5
21	0.06905	...	15.66	65.13	

314.9				
37	0.05863	...	22.81	84.46
545.9				
46	0.06503	...	21.96	57.26
242.2				

	worst smoothness	worst compactness	worst concavity	\
19	0.14400	0.17730	0.23900	
20	0.13120	0.27760	0.18900	
21	0.13240	0.11480	0.08867	
37	0.09701	0.04619	0.04833	
46	0.12970	0.13570	0.06880	

	worst concave points	worst symmetry	worst fractal dimension
diagnosis			
19	0.12880	0.2977	0.07259
1			
20	0.07283	0.3184	0.08183
1			
21	0.06227	0.2450	0.07773
1			
37	0.05013	0.1987	0.06169
1			
46	0.02564	0.3105	0.07409
1			

[5 rows x 31 columns]

Negative diagnosis (Benign):

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
\					
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry
\				
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069

```

3          0.28390          0.2414          0.10520
0.2597
4          0.13280          0.1980          0.10430
0.1809

    mean fractal dimension ... worst texture worst perimeter worst
area \
0          0.07871 ...          17.33          184.60
2019.0
1          0.05667 ...          23.41          158.80
1956.0
2          0.05999 ...          25.53          152.50
1709.0
3          0.09744 ...          26.50          98.87
567.7
4          0.05883 ...          16.67          152.20
1575.0

    worst smoothness worst compactness worst concavity worst concave
points \
0          0.1622          0.6656          0.7119
0.2654
1          0.1238          0.1866          0.2416
0.1860
2          0.1444          0.4245          0.4504
0.2430
3          0.2098          0.8663          0.6869
0.2575
4          0.1374          0.2050          0.4000
0.1625

    worst symmetry worst fractal dimension diagnosis
0          0.4601          0.11890          0
1          0.2750          0.08902          0
2          0.3613          0.08758          0
3          0.6638          0.17300          0
4          0.2364          0.07678          0

[5 rows x 31 columns]

```

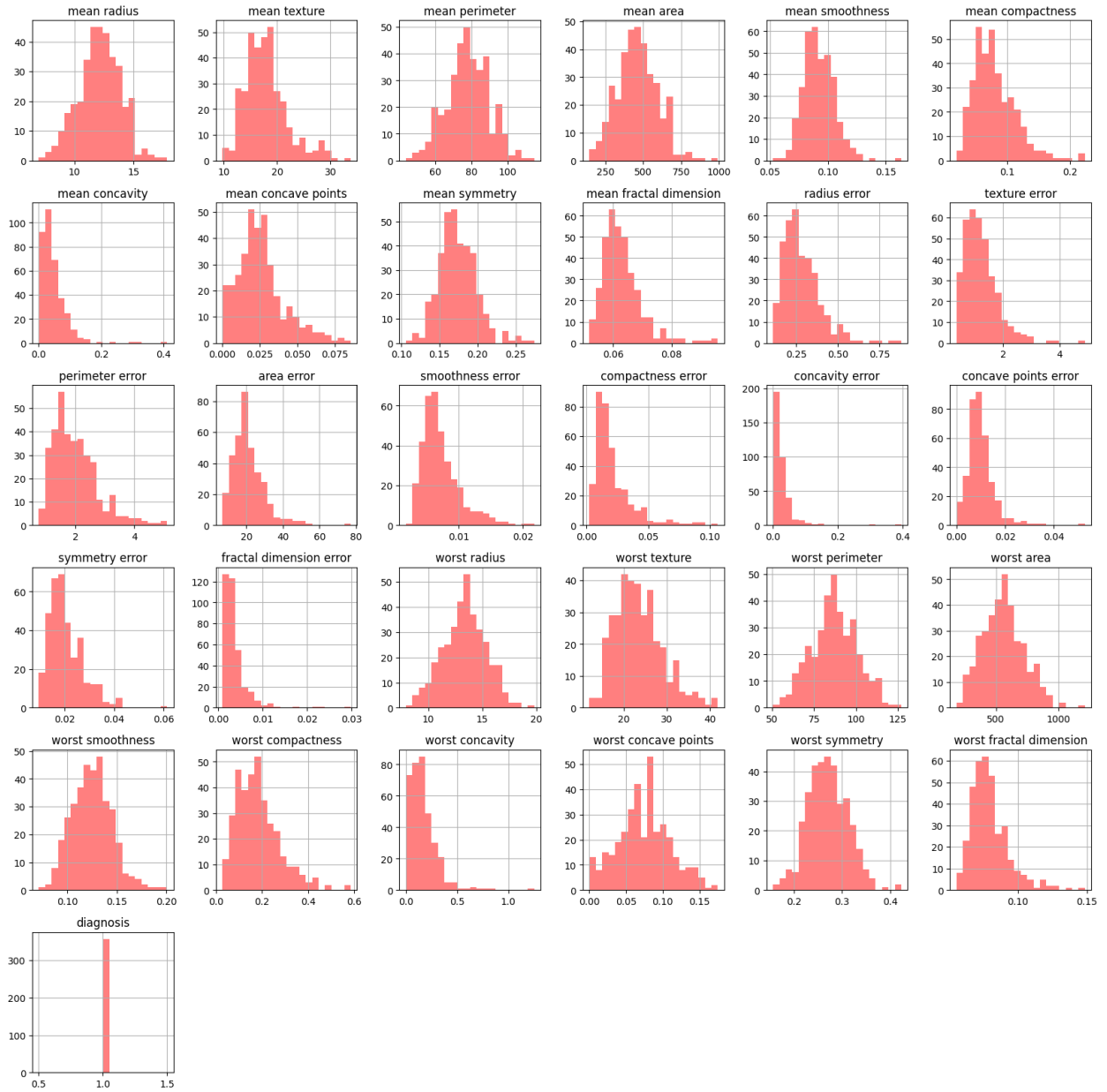
Q6. Visualize tumor characteristics for positive and negatives diagnoses

```

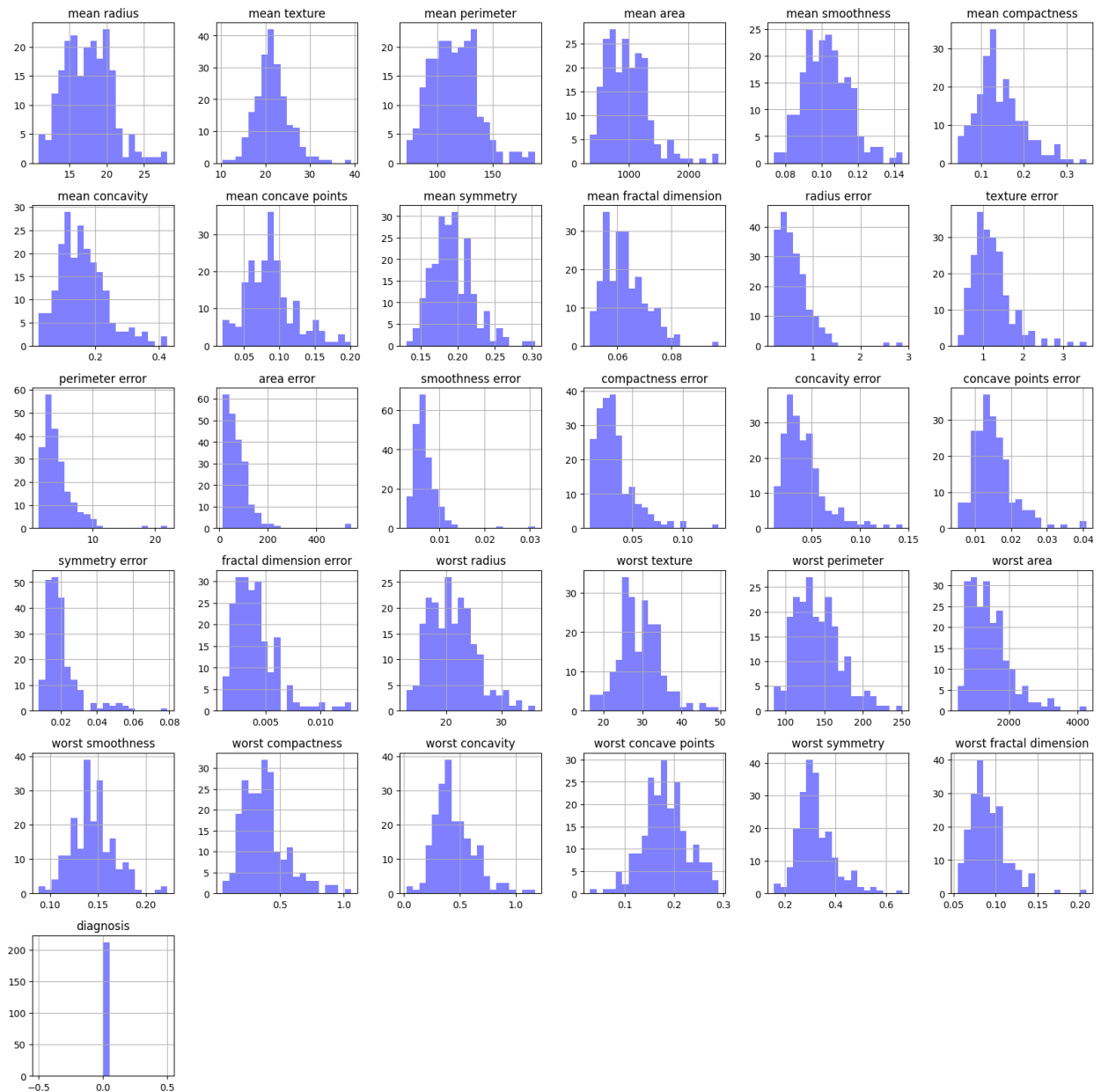
import matplotlib.pyplot as plt
df_positive.hist(figsize=(20, 20), bins=20, color='red', alpha=0.5)
plt.suptitle('Positive Diagnosis (Malignant)')
plt.show()
df_negative.hist(figsize=(20, 20), bins=20, color='blue', alpha=0.5)
plt.suptitle('Negative Diagnosis (Benign)')
plt.show()

```

Positive Diagnosis (Malignant)



Negative Diagnosis (Benign)



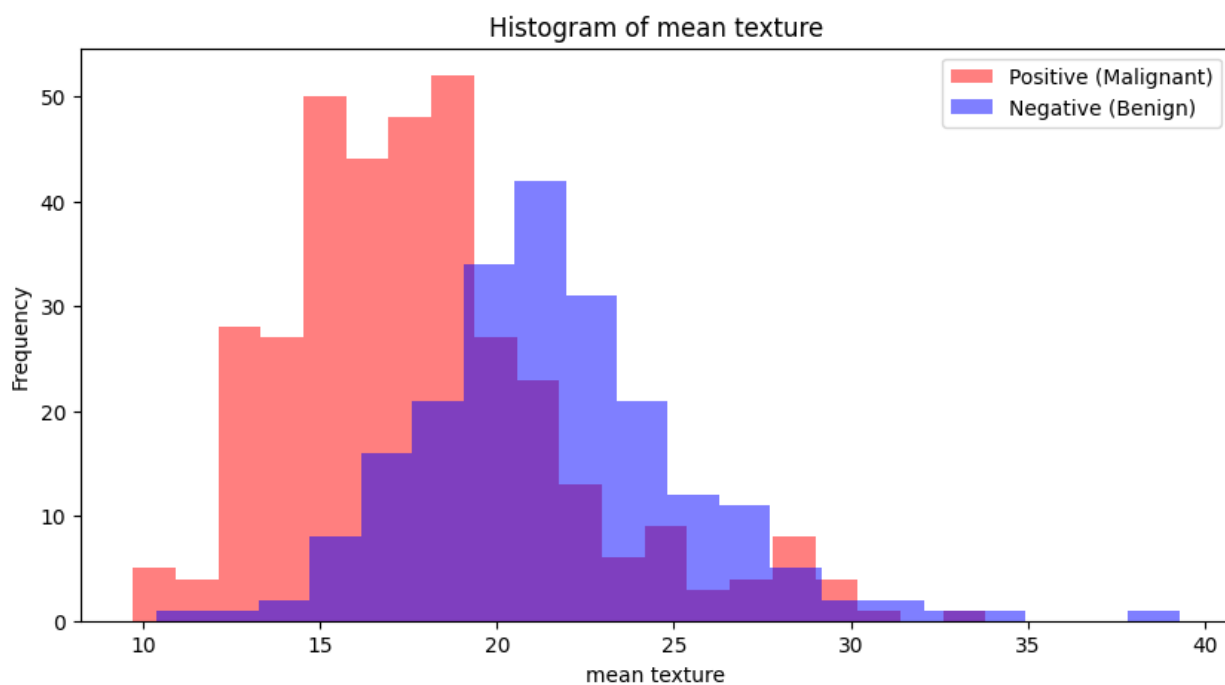
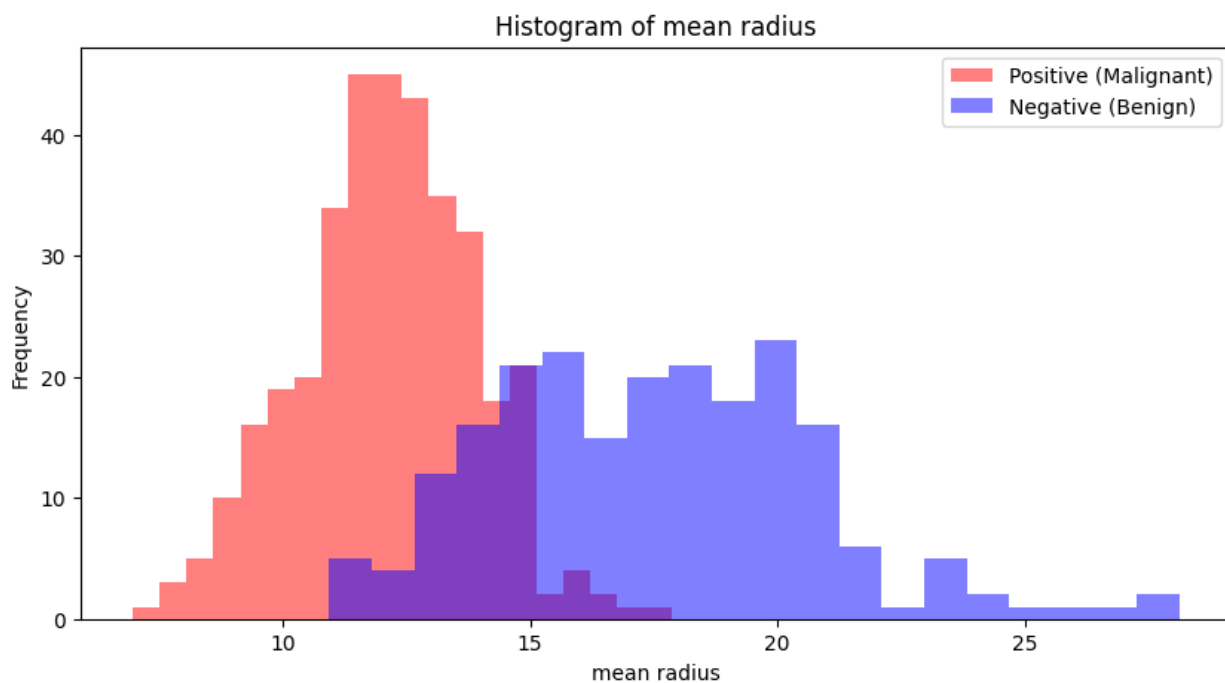
Q7. Create 'for loop' to enerate though tumor features and compare with histograms(20:-1)

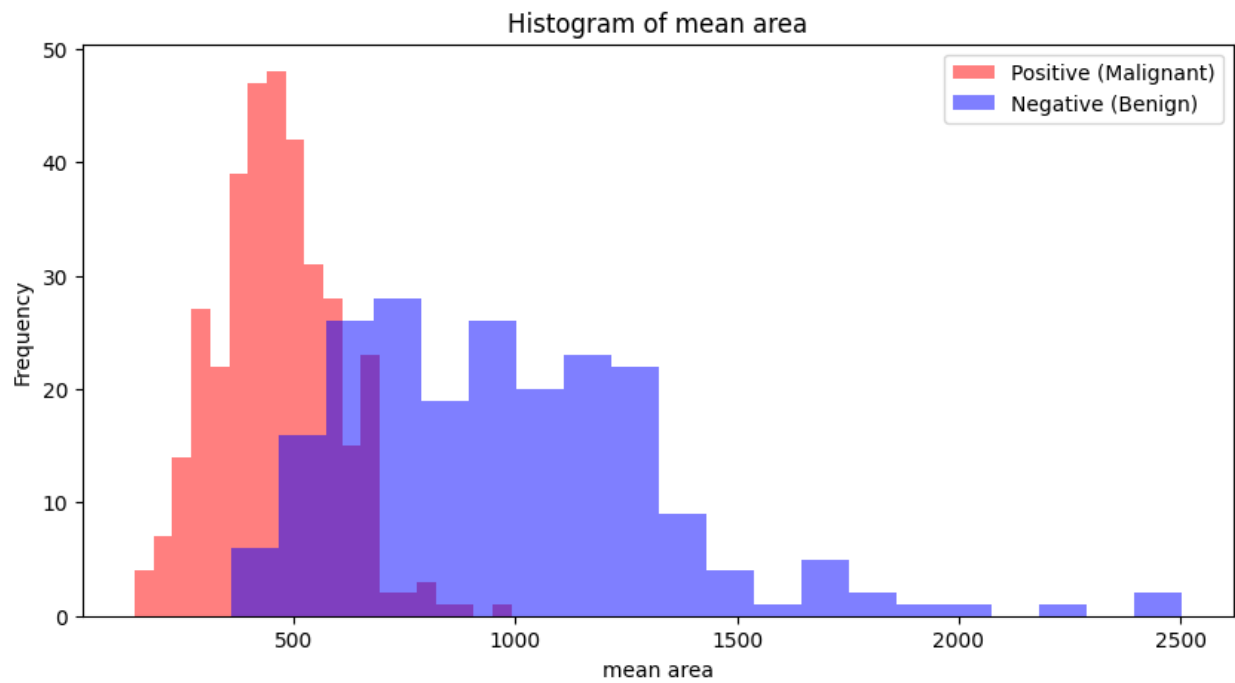
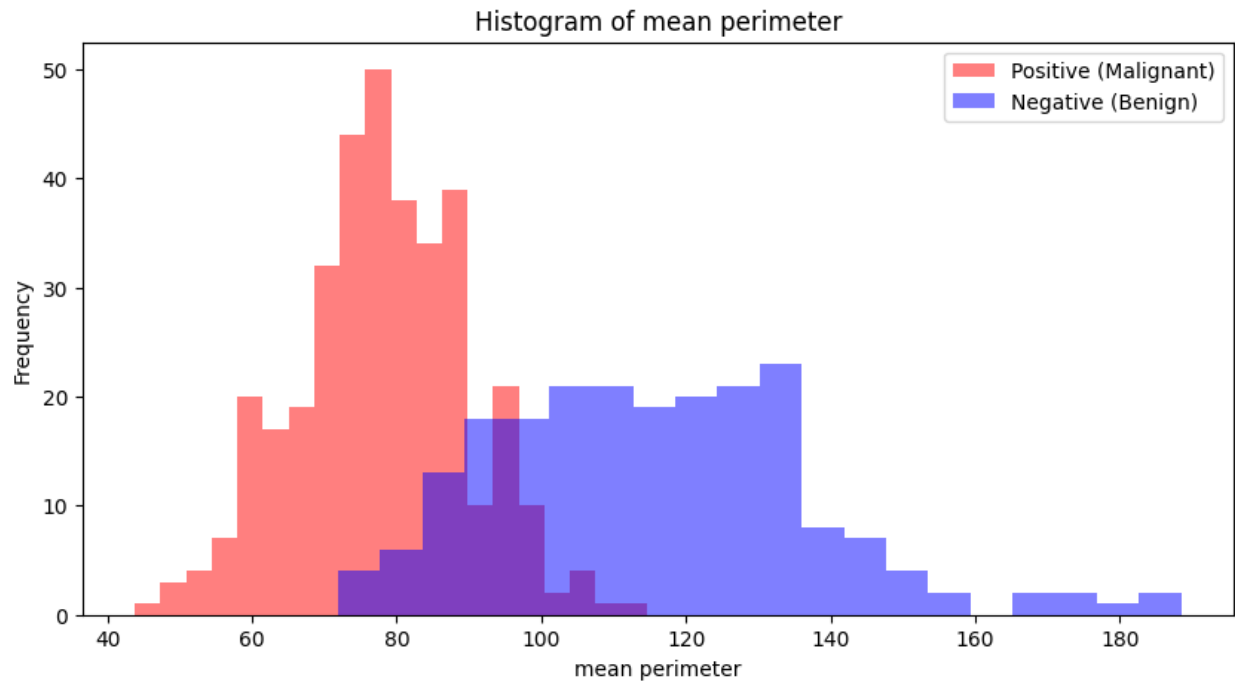
```
features = df_features.columns
for feature in features:
    plt.figure(figsize=(10, 5))
    plt.hist(df_positive[feature], bins=20, alpha=0.5, label='Positive (Malignant)', color='red')
    plt.hist(df_negative[feature], bins=20, alpha=0.5, label='Negative
```

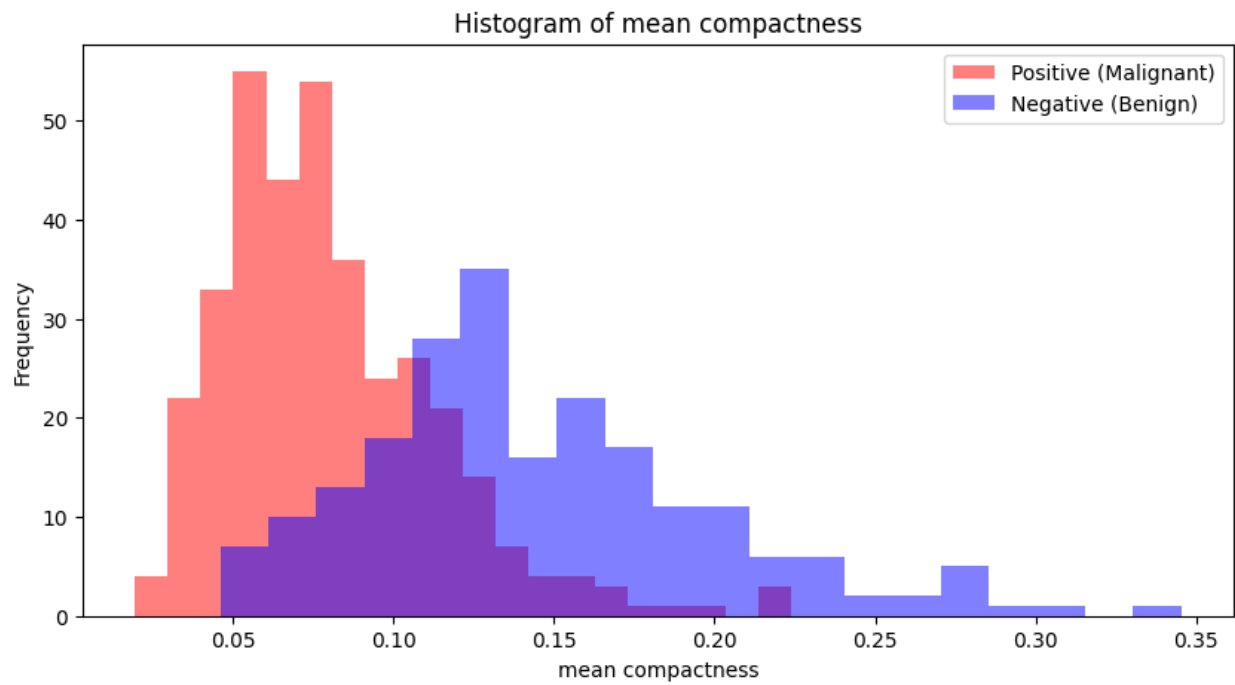
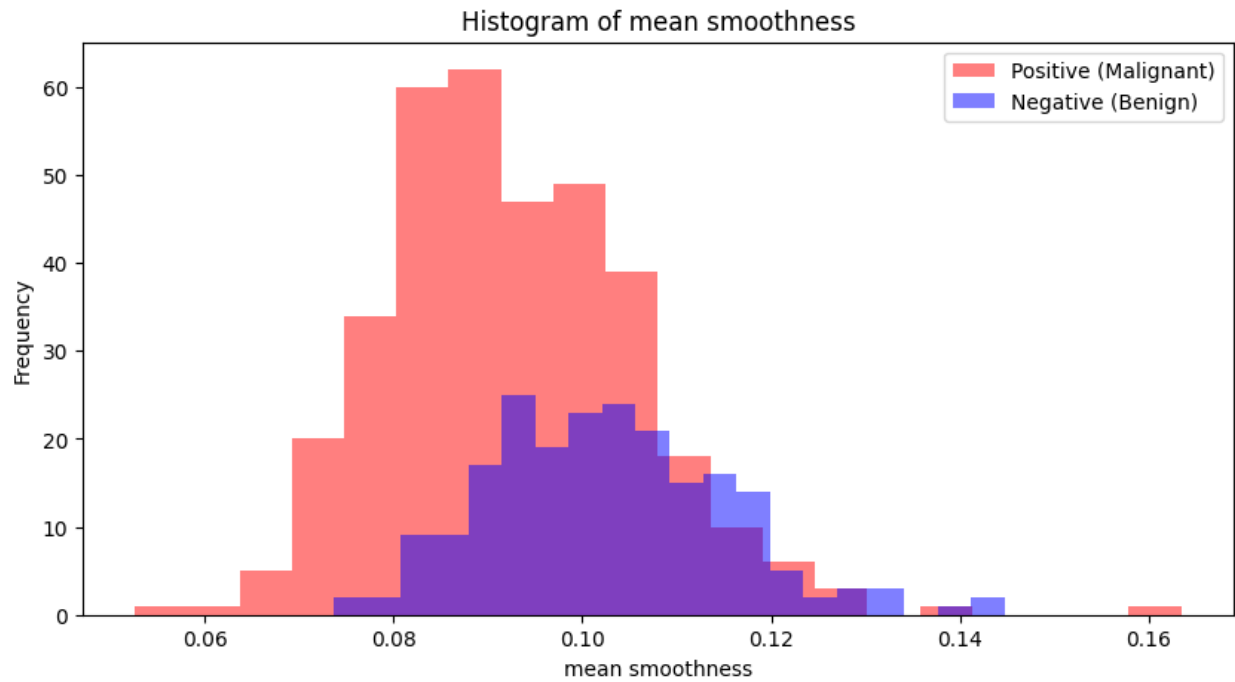
```

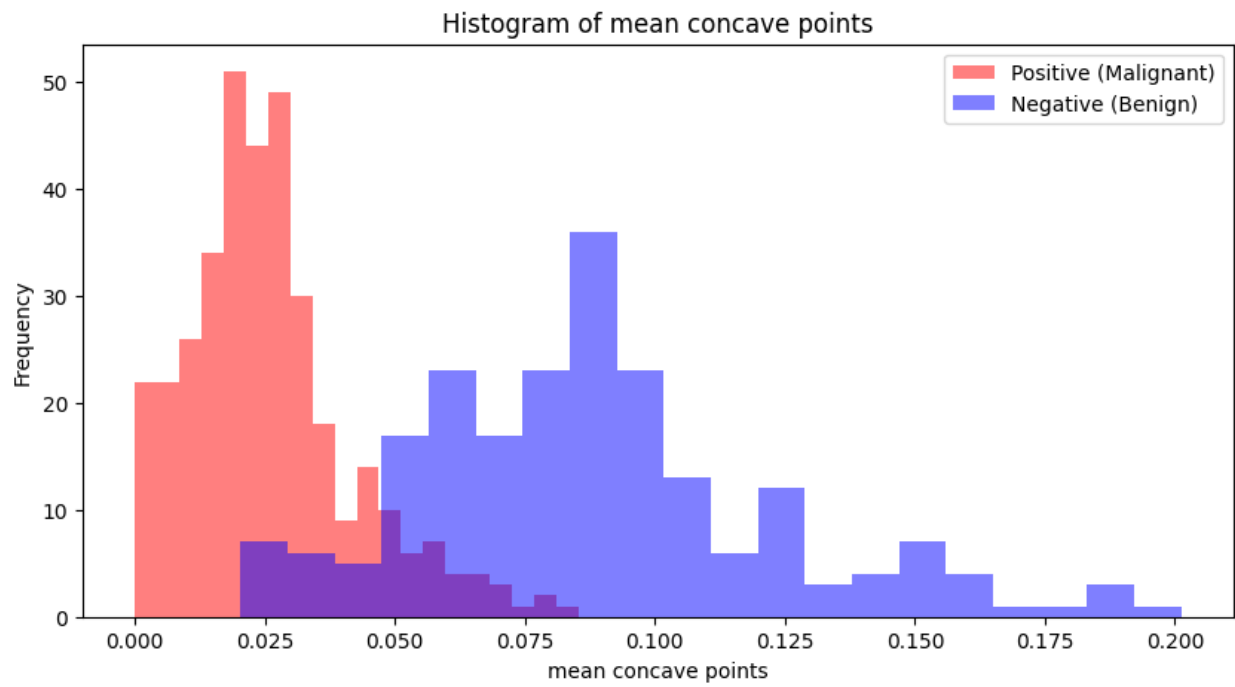
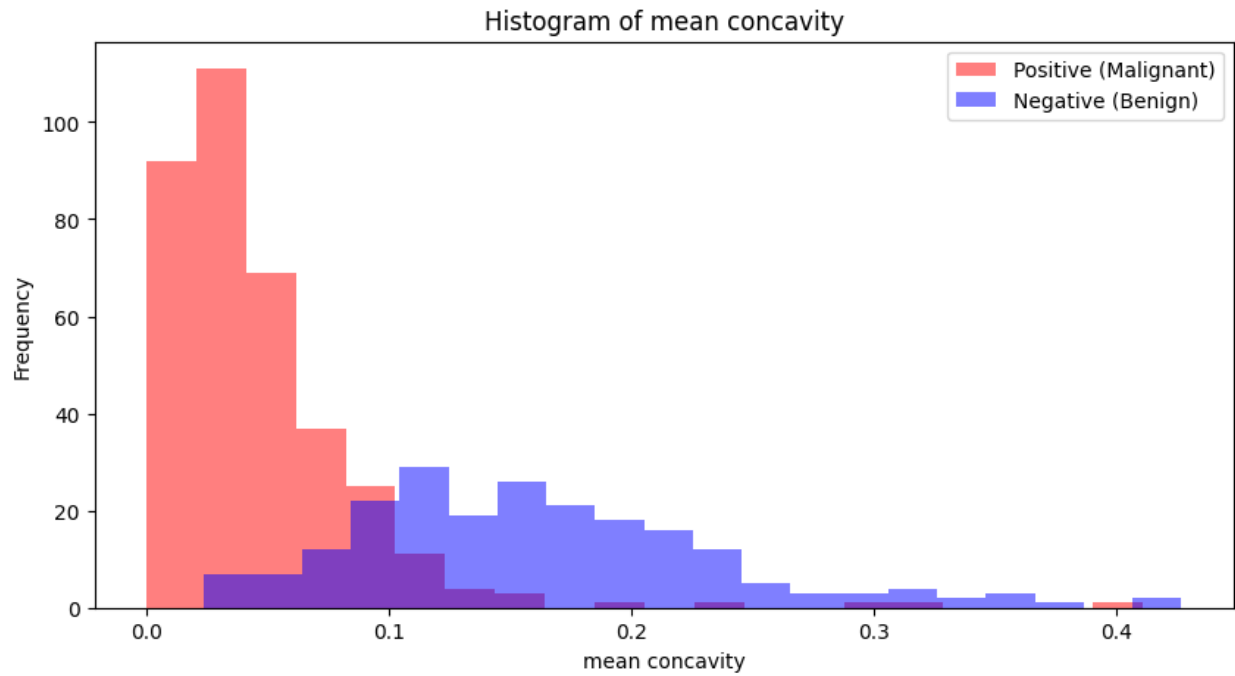
(Benign)', color='blue')
plt.title(f'Histogram of {feature}')
plt.xlabel(feature)
plt.ylabel('Frequency')
plt.legend()
plt.show()

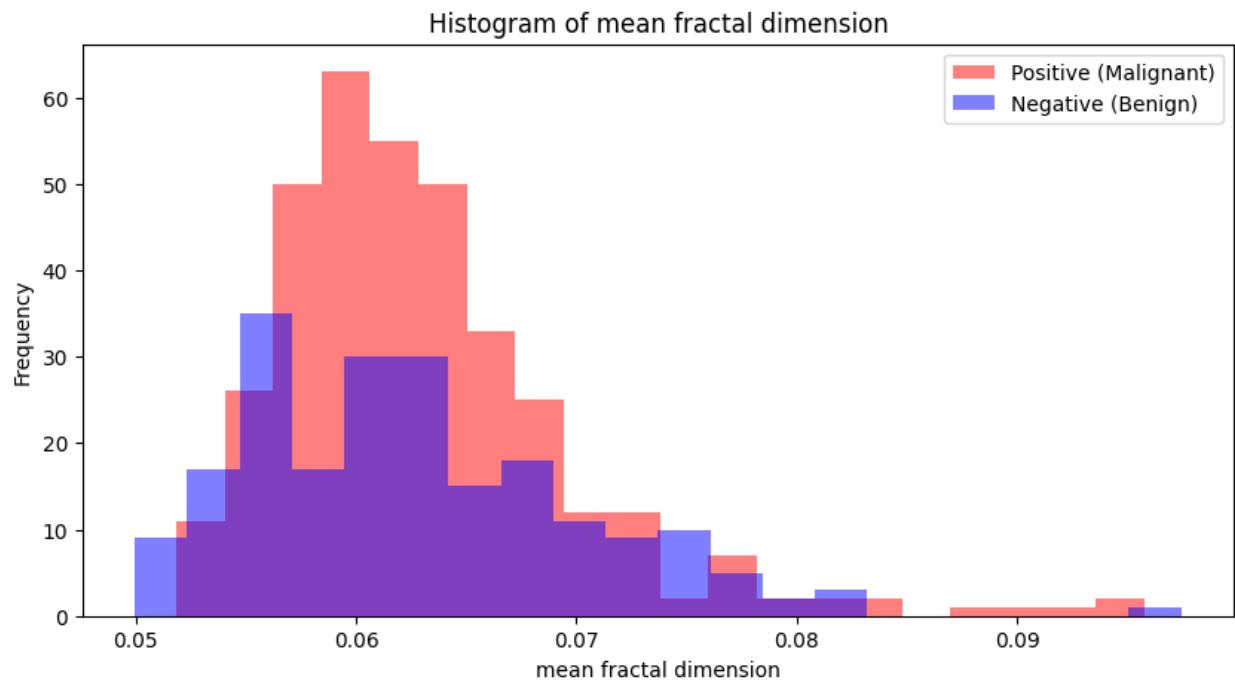
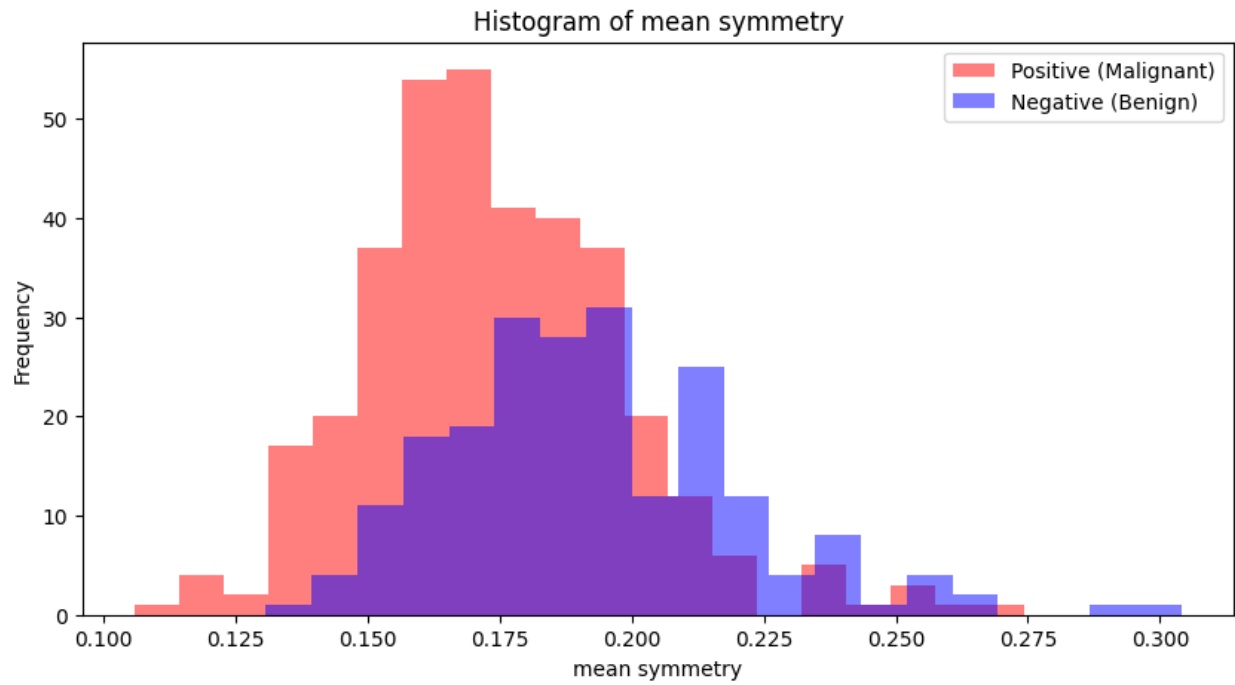
```

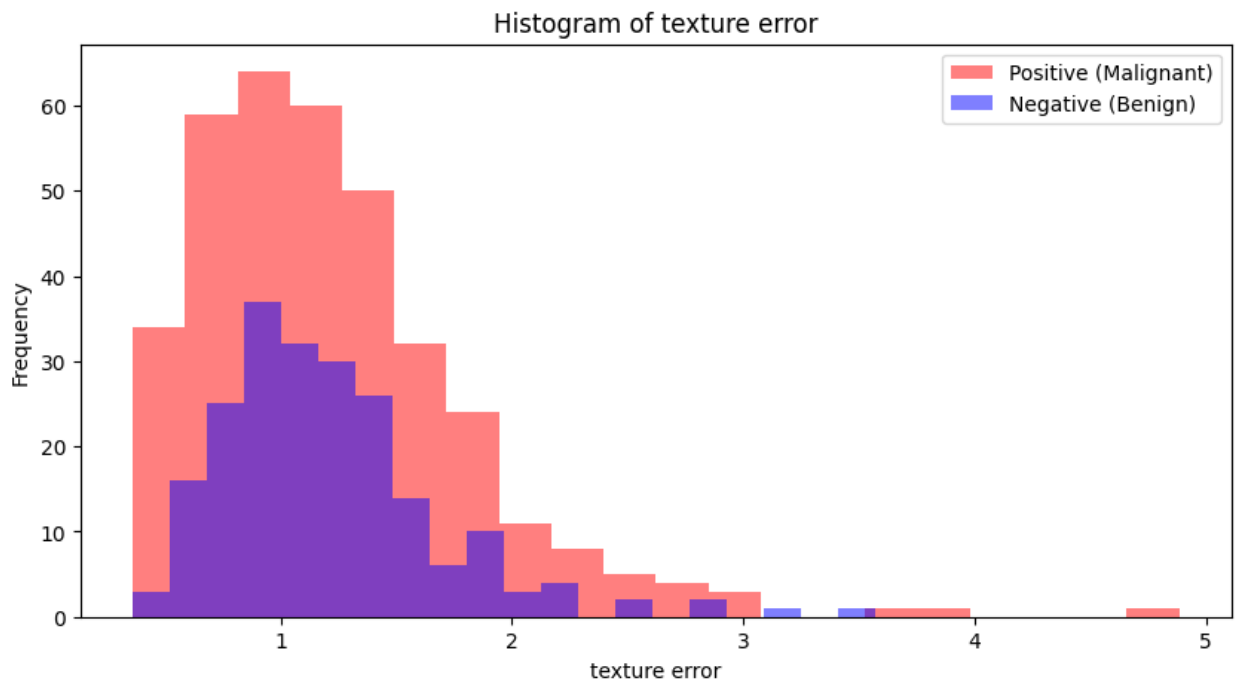
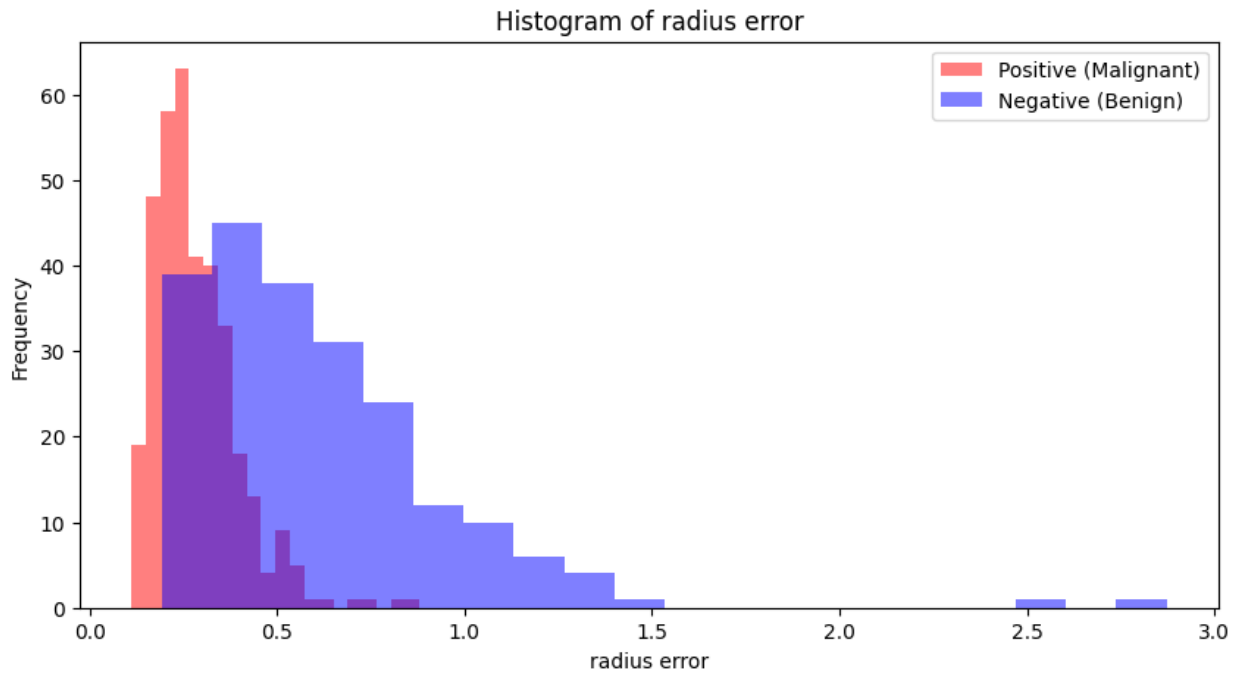


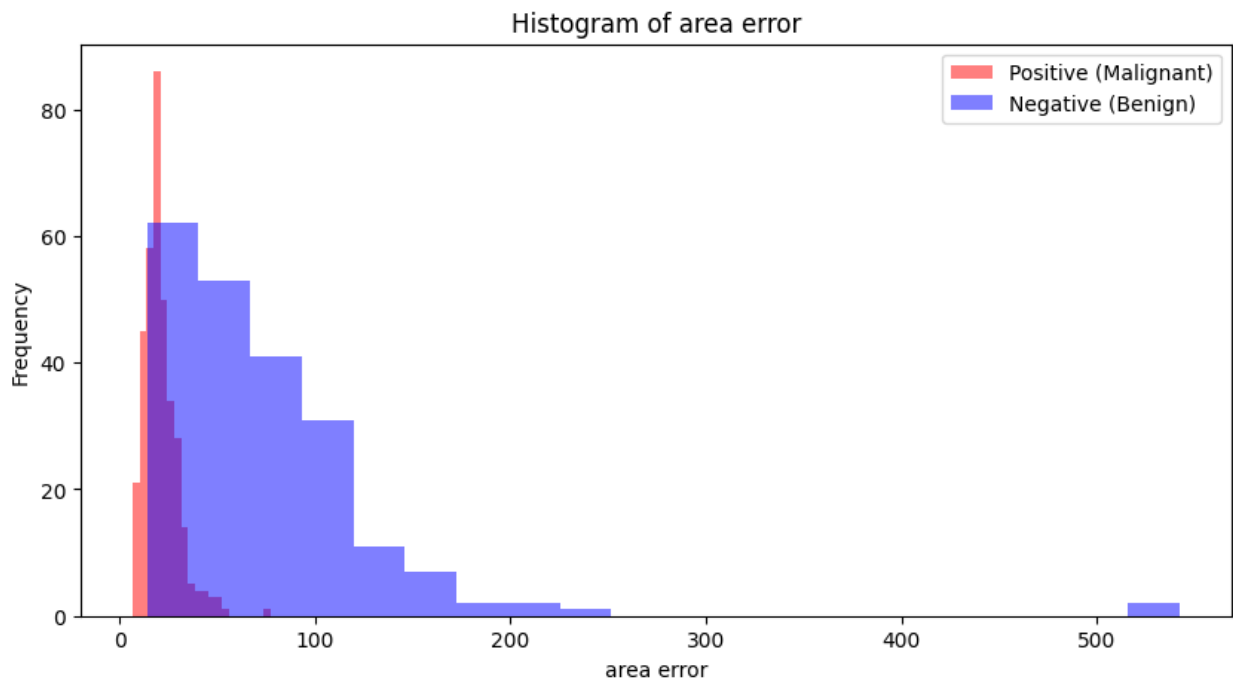


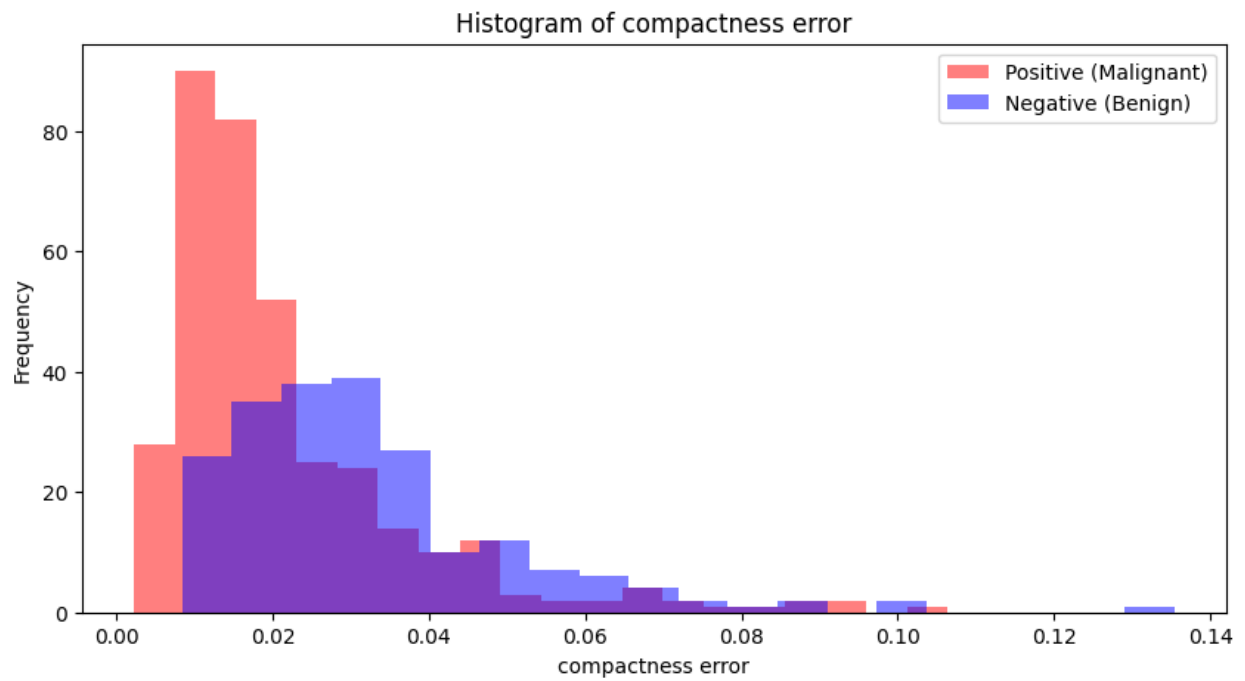
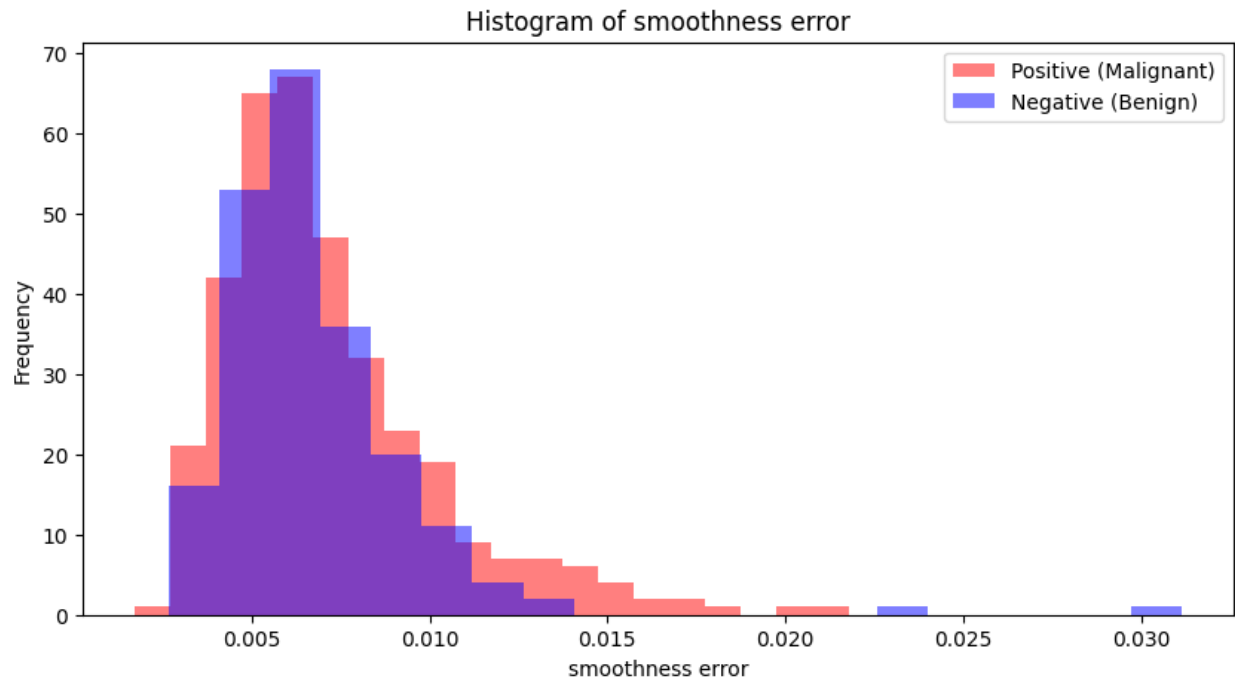


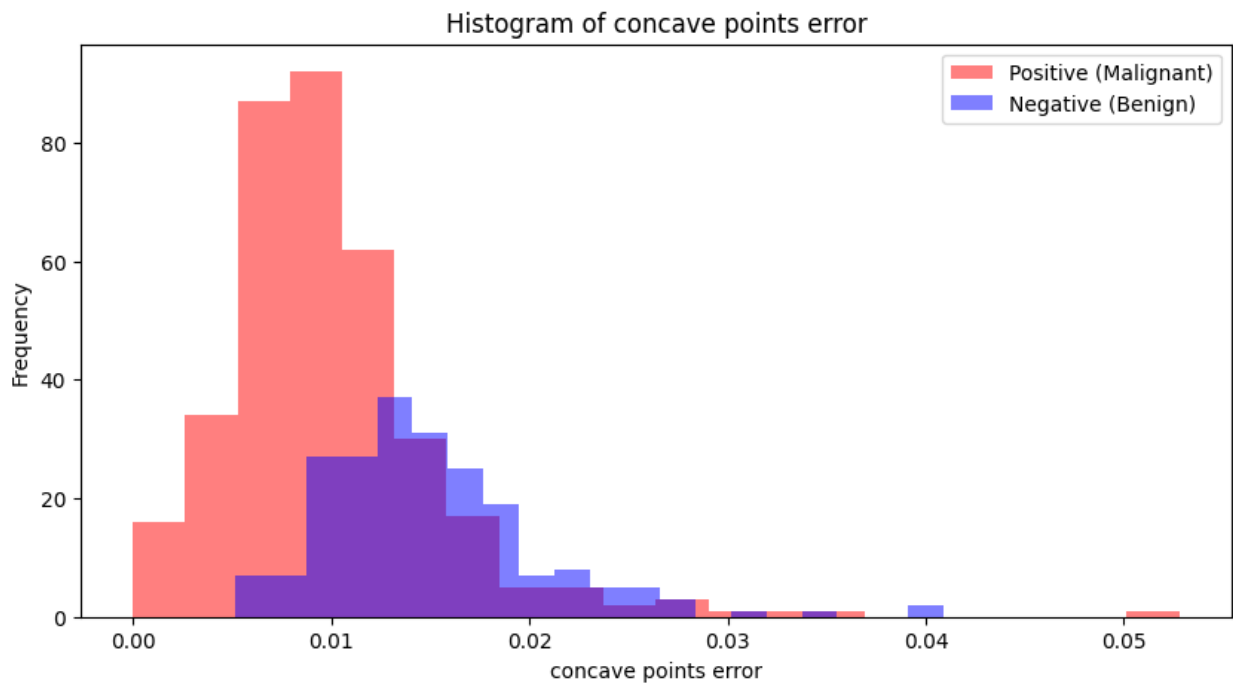
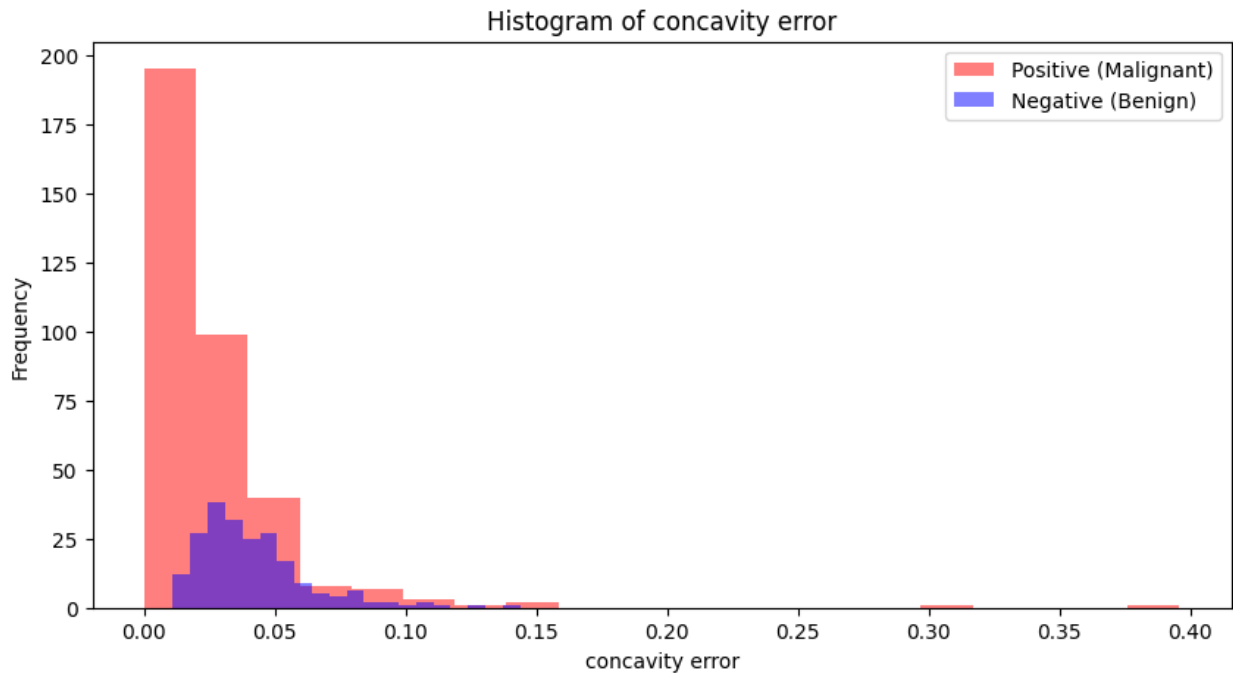


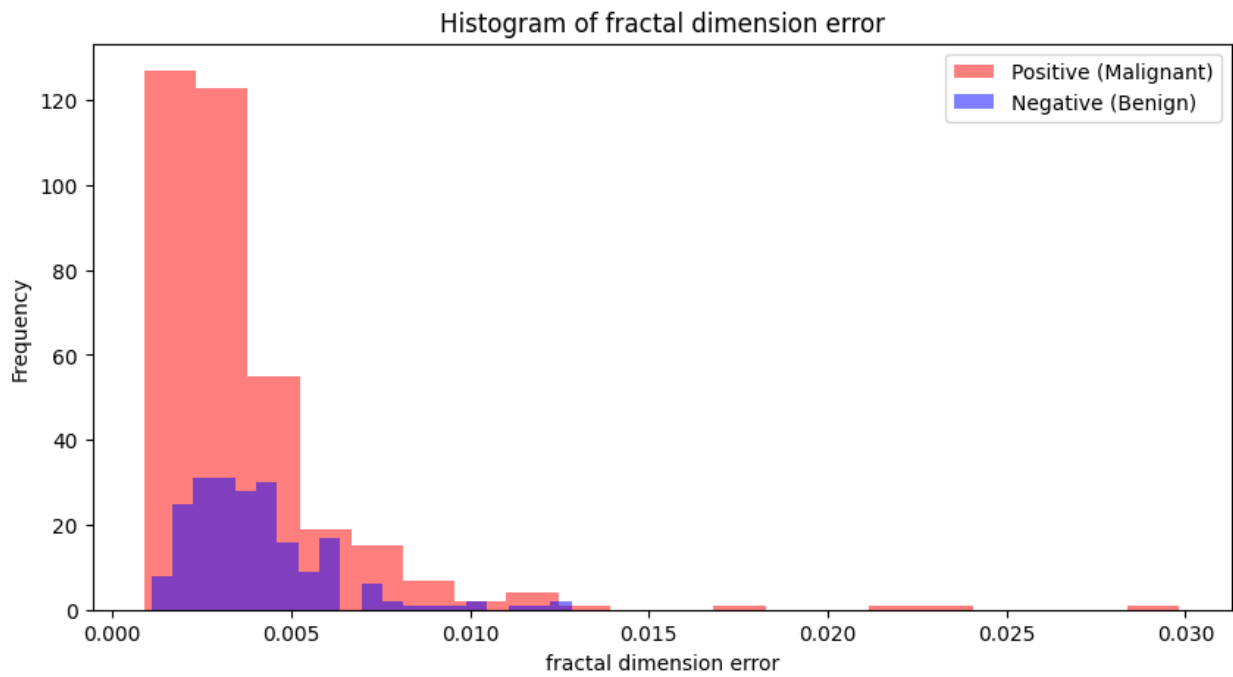
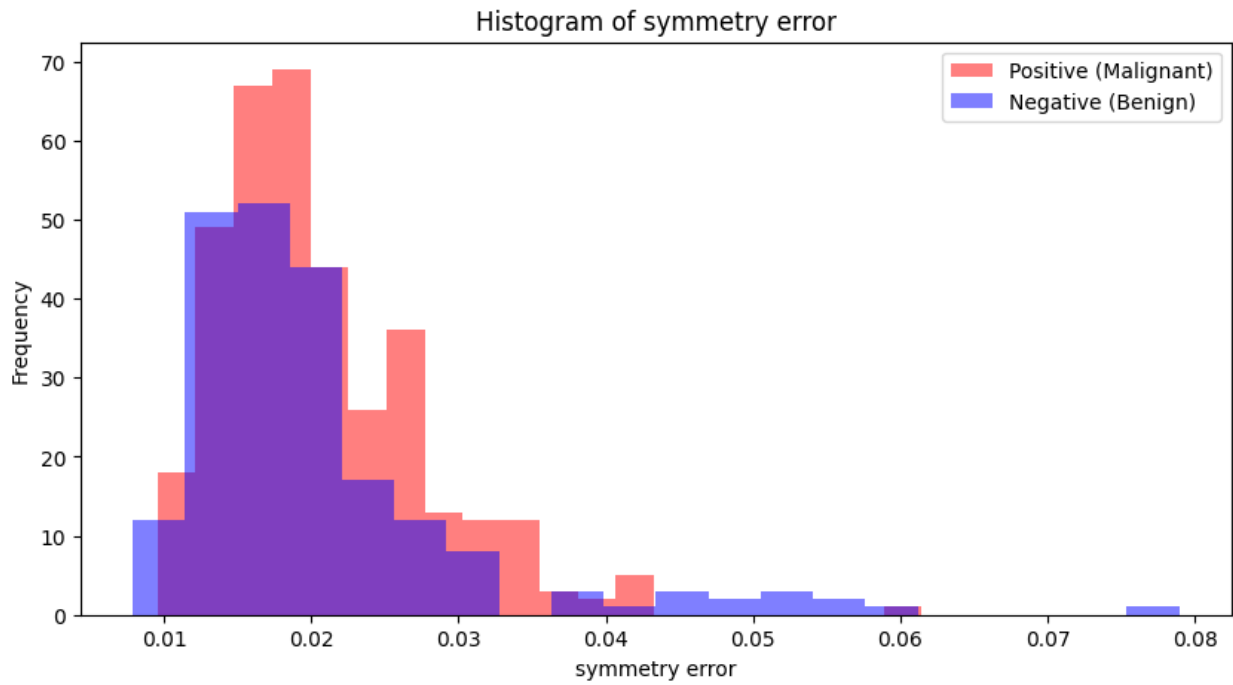


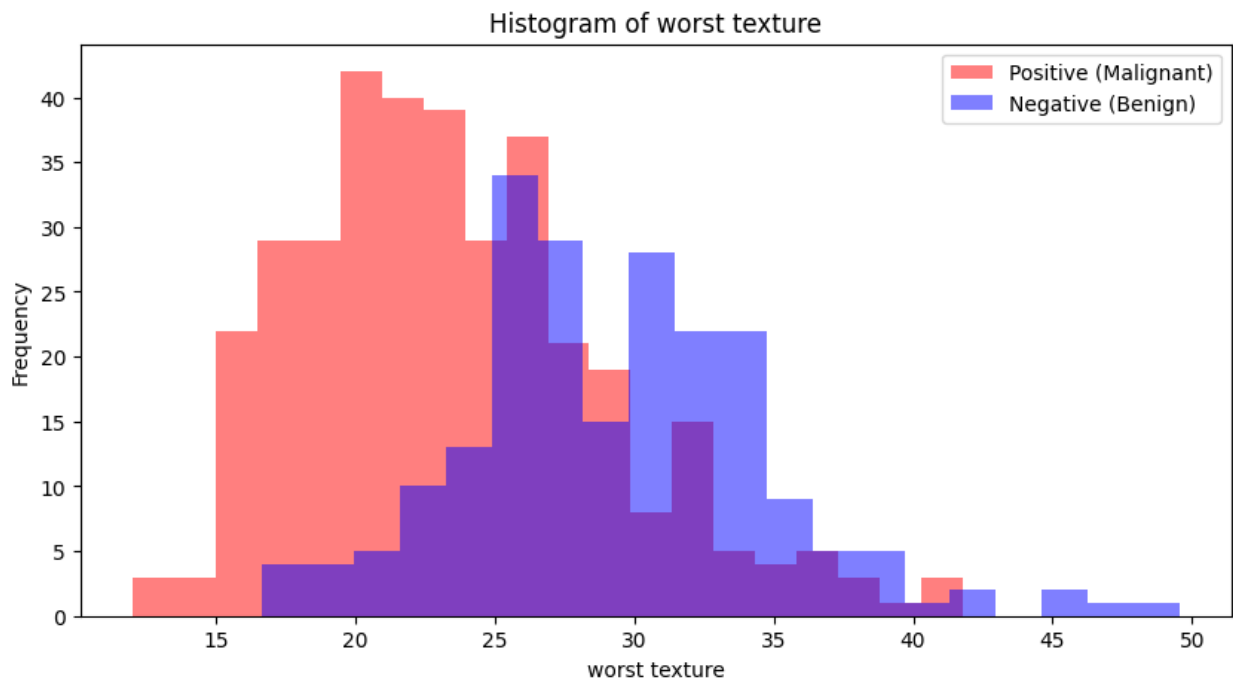
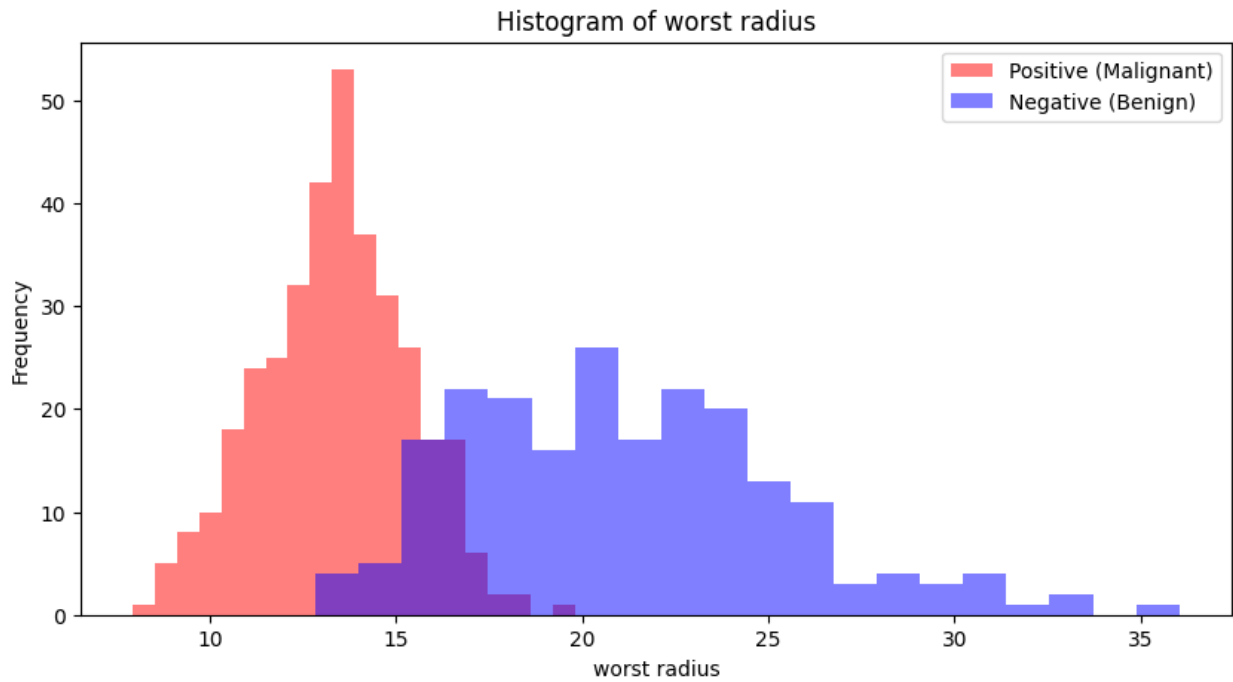


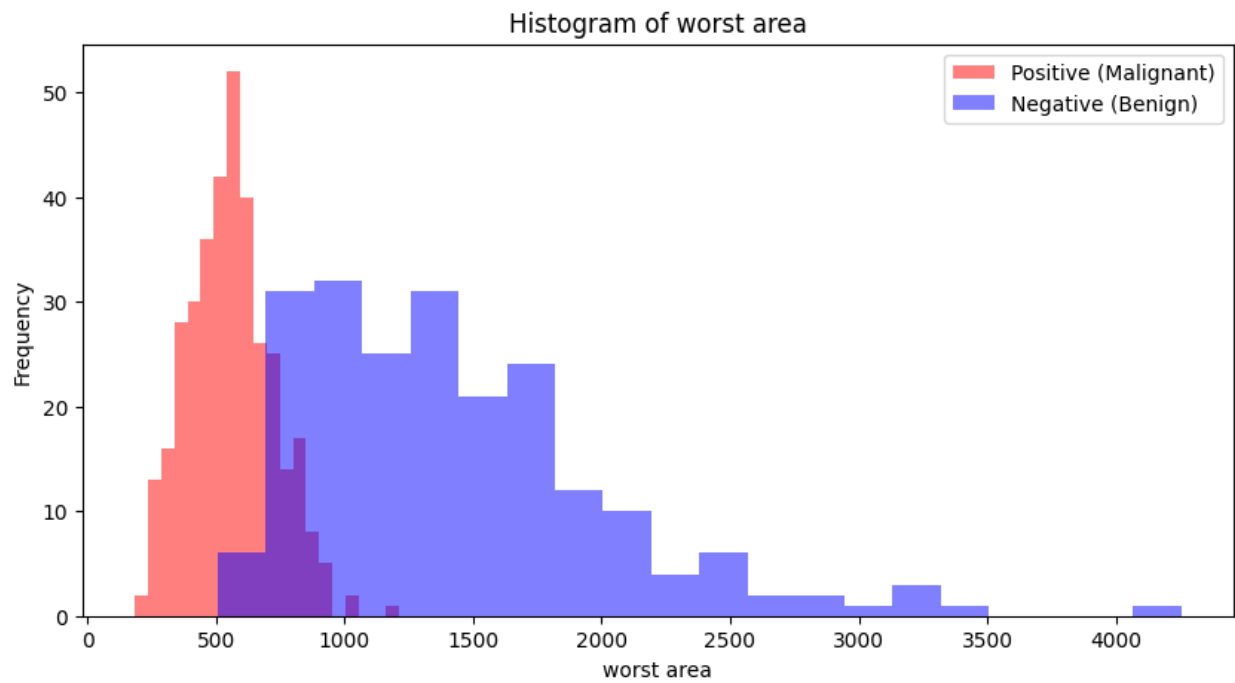
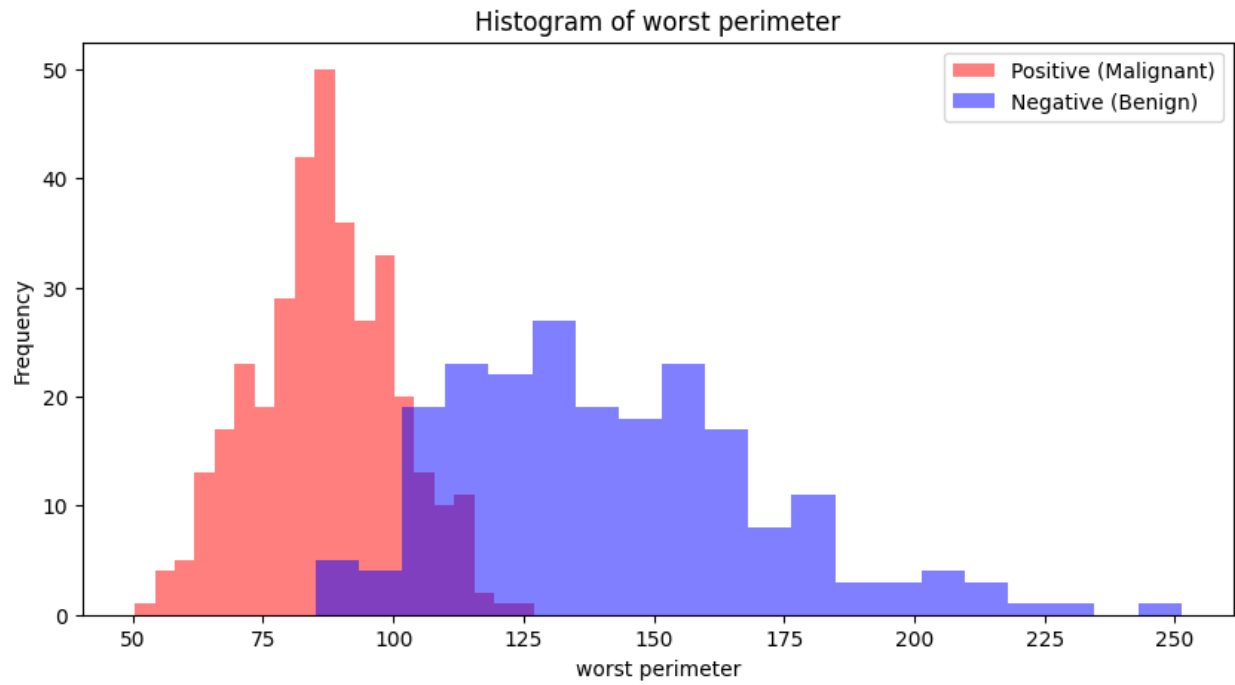


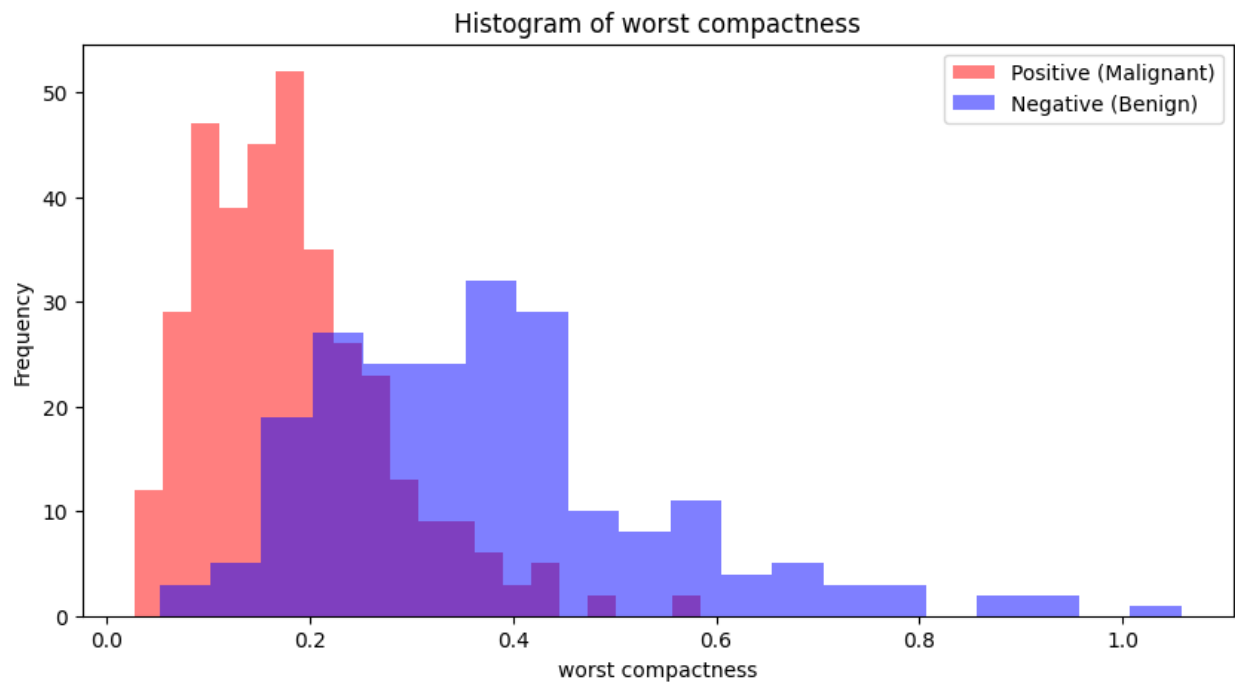
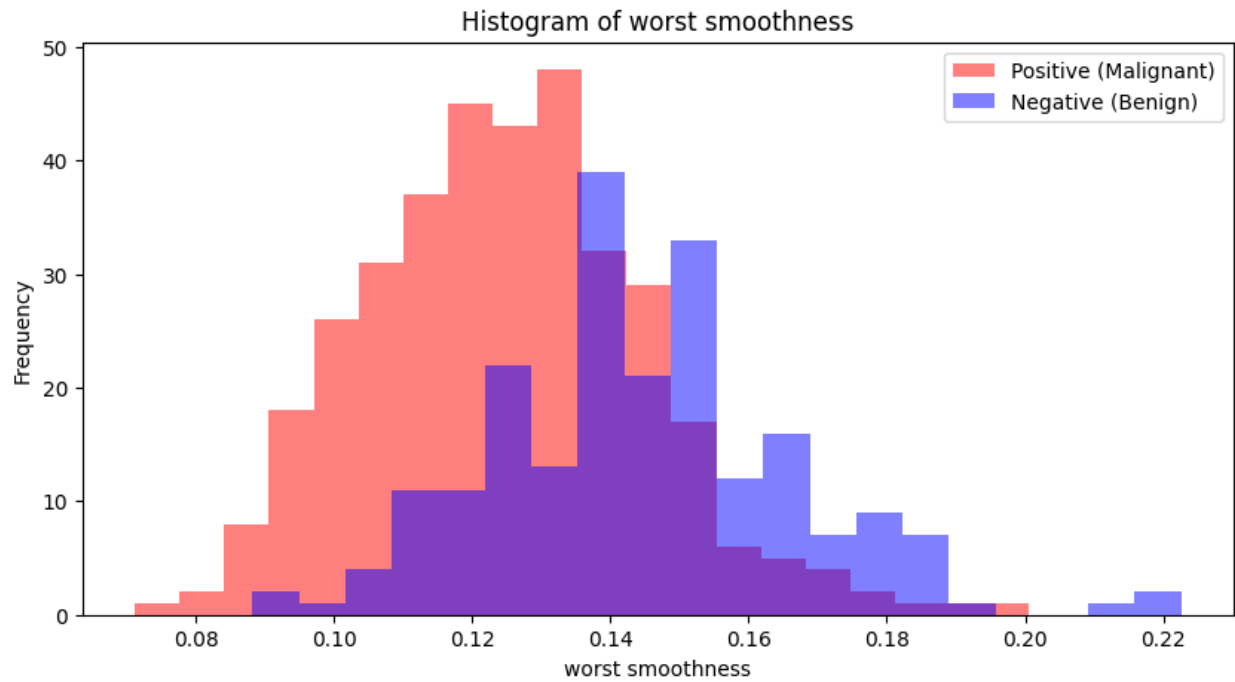


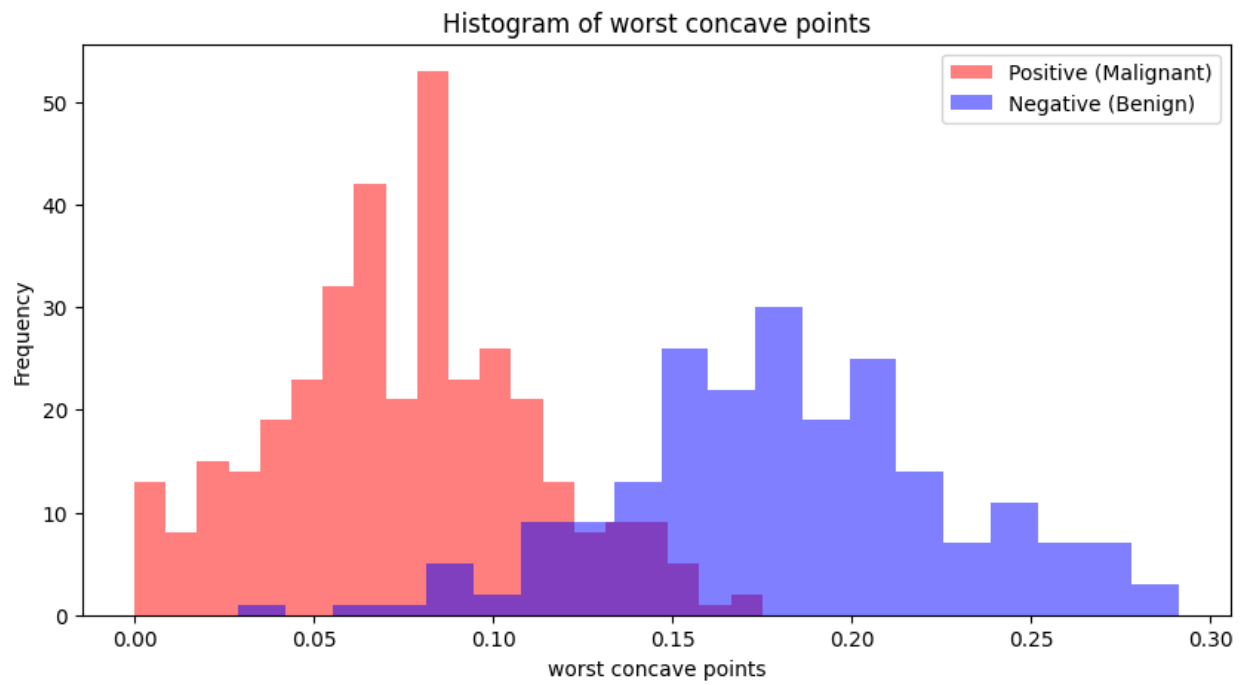
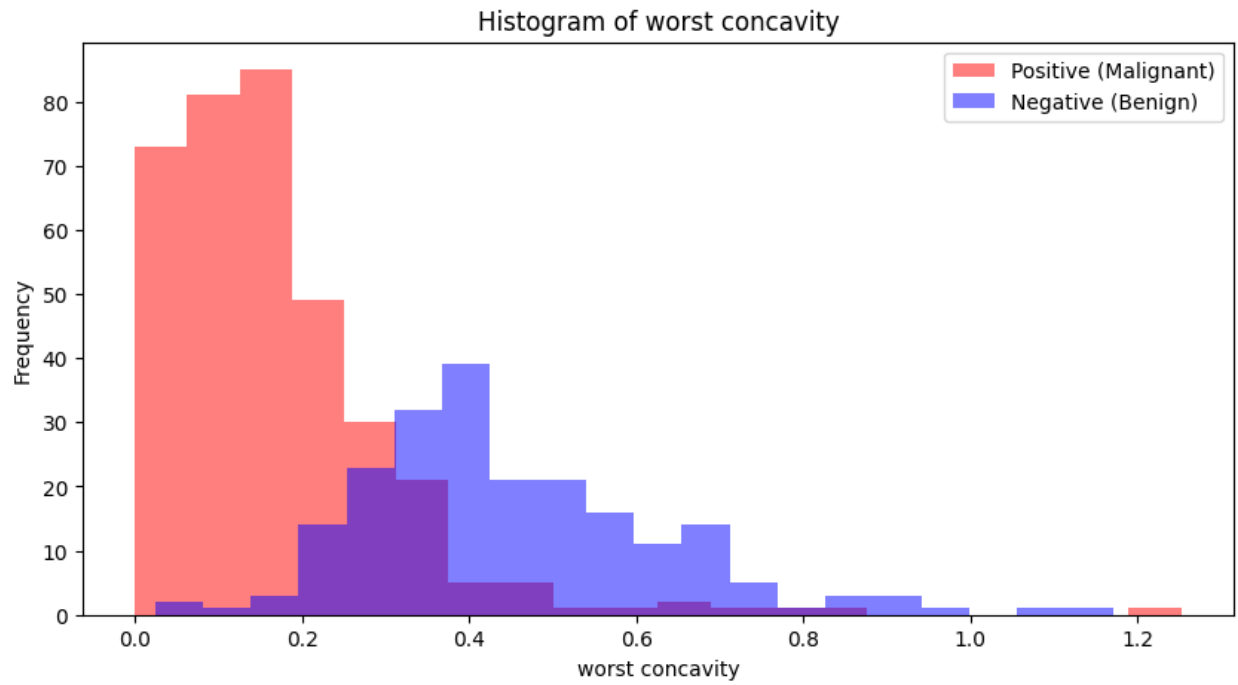


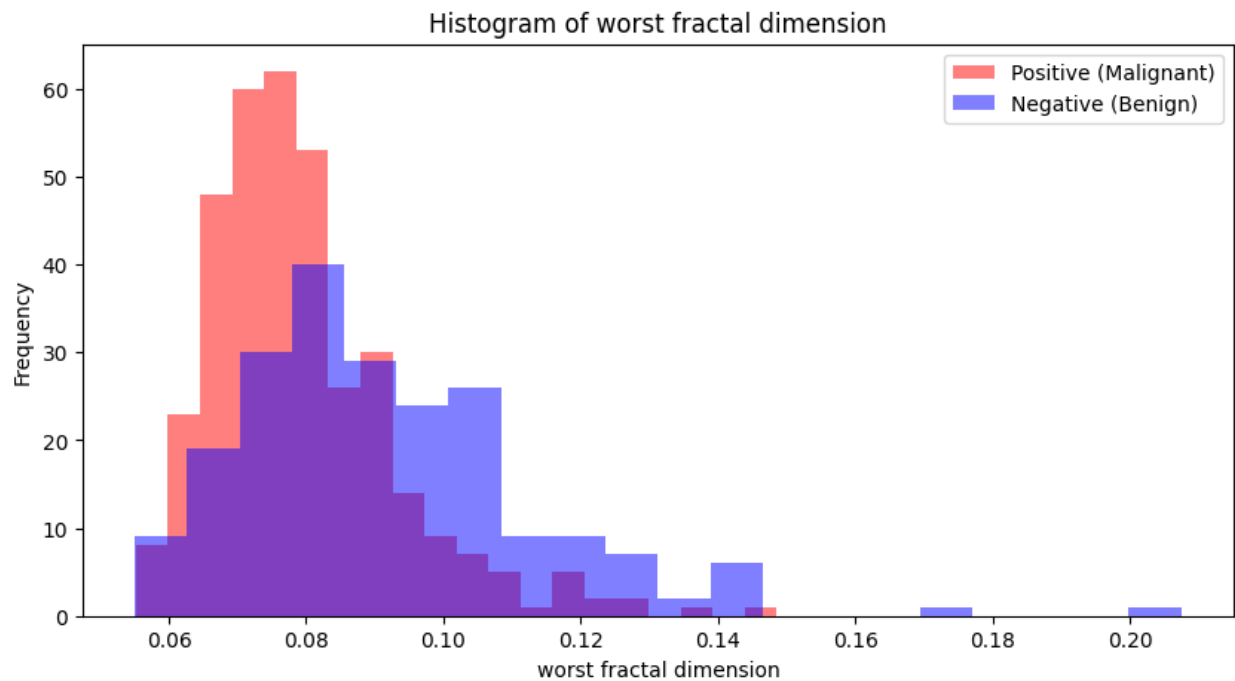
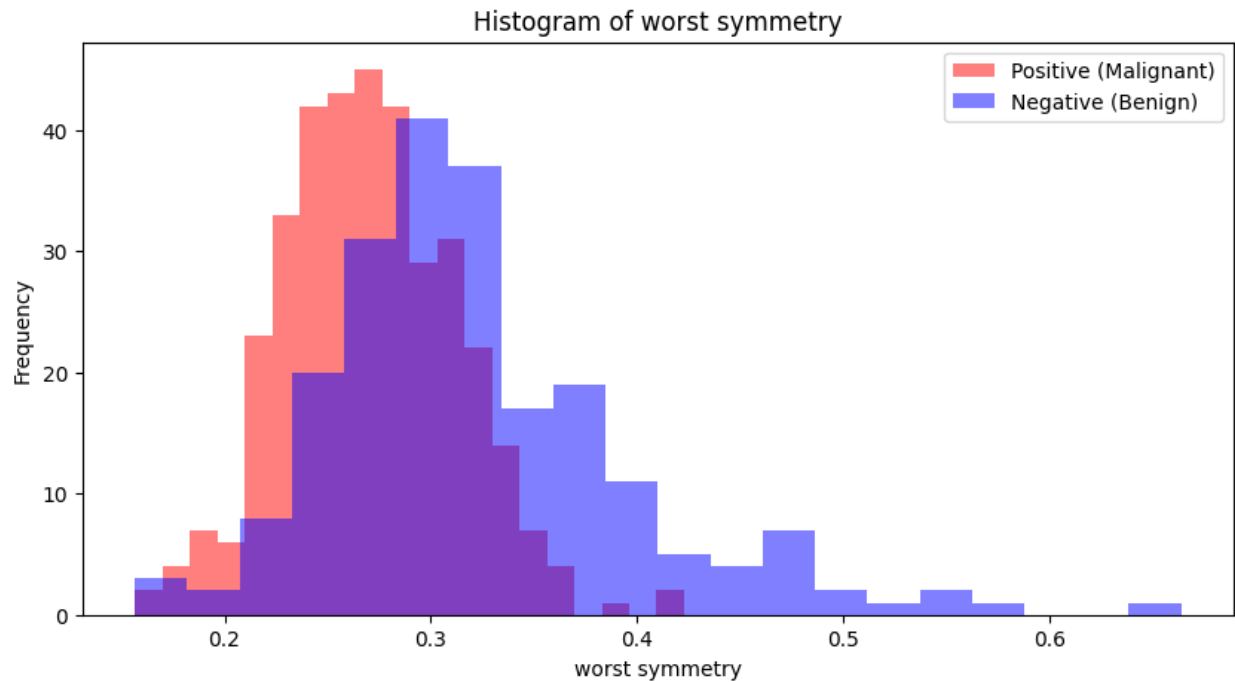






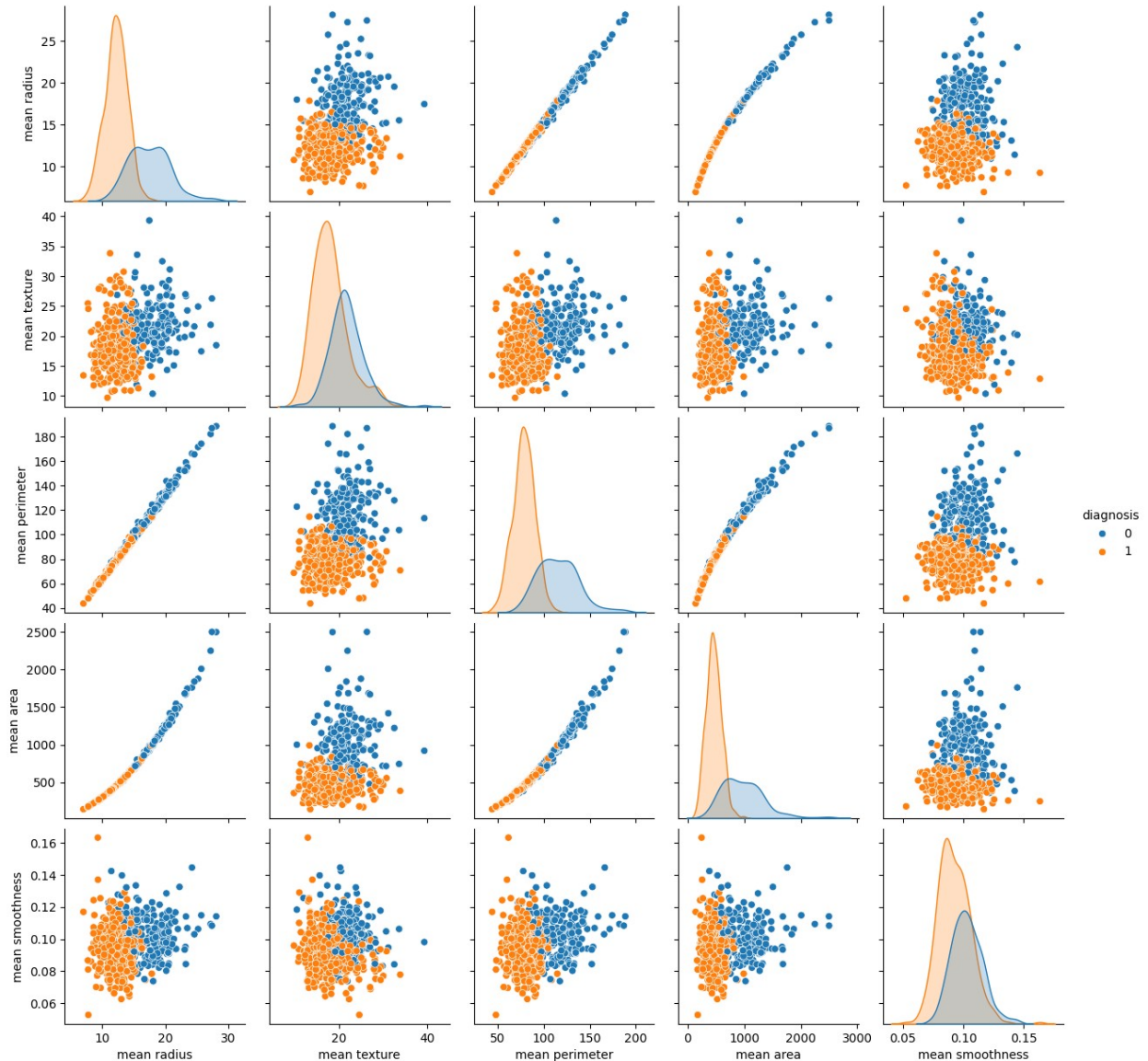






Q8. Find Visualization of relationships between features and diagnoses.

```
import seaborn as sns
sns.pairplot(df, hue='diagnosis', vars=features[:5])
plt.show()
```



Q9. Split data into testing and training set. Use 80% for training

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)
```

Training set shape: (455, 30)
Testing set shape: (114, 30)

Q10. normalize features to account for feature scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Q11. Apply DT, NB, KNN Logistic Regression and SVM to find accuracy, F1-Score and Recall.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, recall_score

models = {
    'Decision Tree': DecisionTreeClassifier(),
    'Naive Bayes': GaussianNB(),
    'K-Nearest Neighbors': KNeighborsClassifier(),
    'Logistic Regression': LogisticRegression(max_iter=10000),
    'Support Vector Machine': SVC()
}

results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    results[name] = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred)
    }

for name, metrics in results.items():
    print(f"{name}:")
    for metric, value in metrics.items():
        print(f"    {metric}: {value:.4f}")
```

```
C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-
packages\joblib\externals\loky\backend\context.py:136: UserWarning:
Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this
warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want
to use.
```

```
warnings.warn(
    File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\site-
packages\joblib\externals\loky\backend\context.py", line 257, in
_count_physical_cores
    cpu_info = subprocess.run(
    File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\
subprocess.py", line 505, in run
    with Popen(*popenargs, **kwargs) as process:
    File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\
subprocess.py", line 951, in __init__
```

```
self._execute_child(args, executable, preexec_fn, close_fds,  
File "C:\Users\Asus\AppData\Local\Programs\Python\Python39\lib\  
subprocess.py", line 1420, in _execute_child  
hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
```

Decision Tree:

Accuracy: 0.9386

F1 Score: 0.9504

Recall: 0.9437

Naive Bayes:

Accuracy: 0.9649

F1 Score: 0.9722

Recall: 0.9859

K-Nearest Neighbors:

Accuracy: 0.9474

F1 Score: 0.9577

Recall: 0.9577

Logistic Regression:

Accuracy: 0.9737

F1 Score: 0.9790

Recall: 0.9859

Support Vector Machine:

Accuracy: 0.9825

F1 Score: 0.9861

Recall: 1.0000

Q12. Further apply the PCA to do the same exercise in Q11.

Q13. Do the Comparative study **with** PCA **and** **With**out PCA.